Live Technical Assessment

Fullstack Engineer (Node.js + React) — Backend-Heavy

Velvetech • 60–90 minutes • Screen sharing required

Candidate Instructions

Format: Live coding session via video call, 60–90 minutes

Camera: Keep your camera on throughout the session

Screen: Share your screen from the start and keep it shared

Tools: You may use any tools — IDE, documentation, AI assistants (Copilot, ChatGPT, Claude, etc.)

Important: You are NOT expected to finish everything. Steps 4 and 5 are stretch goals. Finishing Steps 1–3 well is a strong result.

Focus on: Clarity, structure, and explaining your decisions — not speed or completeness

After session: Send your solution as a ZIP archive to i.pukhov@velvetech.dev

README: A short README explaining your decisions is required — see Deliverables section

Business Scenario

You are building a backend service for AssetPulse — an internal platform used by facility managers to monitor the health and status of physical assets (HVAC units, elevators, sensors) across multiple buildings.

Each building belongs to a tenant — a client organization. Facility managers log in and see only their own buildings and assets. Assets periodically push status updates (temperature readings, error codes, operational state), and the platform must reflect this in near real-time.

Your task is to design and partially implement the core backend service for this platform, with an optional real-time layer and a minimal frontend view.

Step-by-Step Structure

Work through these steps in order. Stop when time runs out — partial completion is expected and acceptable.

Step 1 — Data Model & Project Setup (0–15 min)

Design the core domain model. You should end up with at least:

- Tenant — a client organization
- Building — belongs to a tenant
- Asset — belongs to a building (e.g., HVAC unit, elevator)
- AssetEvent — a status update pushed by an asset (timestamp, type, payload)

Use TypeScript interfaces or classes. If you use an ORM (TypeORM, Prisma), define the schema. If not, plain interfaces with a .sql DDL file are fine.

> **Design decision:** Be ready to explain your normalization choices and how you would handle tenant isolation at the query level.

Step 2 — REST API (15–35 min)

Implement the following endpoints using Express or NestJS — your choice.

Required endpoints (2 of 2):

- GET /buildings — return buildings for the authenticated tenant
- POST /assets/:id/events — ingest a new status event for an asset

Optional third endpoint (if time allows):

- GET /buildings/:id/assets — return assets for a building

Rules:

- Tenant context must come from the request (header, JWT claim, or query param — your choice, but explain it)
- Every read/write operation must verify that the requested building/asset belongs to the current tenant
- No real authentication required — simulate tenant context however you prefer
- Validate request bodies — basic validation is enough
- Return consistent, typed JSON responses

> **Design decision:** How do you prevent tenant A from accessing tenant B's data? Show it in code or explain it explicitly.

Step 3 — Business Logic (35–55 min)

This step is required. Implement the following rule as part of the event ingestion flow:

> **Rule:** If an asset receives 3 or more error events within a 10-minute window, mark the asset status as "critical" and log a warning.

Implement this as a named function in a service layer — not inline in the controller. If you run out of time, write the function signature and explain the logic verbally. A clear explanation counts.

> **Simplification allowed:** An in-memory implementation is fine. You do not need a real database query — a plain array of events in memory works perfectly for this.

Step 4 — Real-Time Layer (stretch — 55–75 min)

Add a basic WebSocket or SSE endpoint that pushes new asset events to connected clients in real-time.

- Clients subscribe to a building: subscribe(buildingId)
- When a new event is ingested via POST /assets/:id/events, emit it to subscribers of that building
- Tenant isolation must still apply

NestJS Gateway, ws, socket.io — any approach is fine. Partial implementation with explanation is acceptable.

Step 5 — Minimal React View (stretch — 75–90 min)

Build a simple dashboard component:

- Fetch and display assets for a given building (hardcode the building ID)
- Show asset name, current status, and last event timestamp
- Poll every 10 seconds OR connect to the WebSocket from Step 4

No styling required. Functional correctness matters more than UI.

Priorities

The assessment is intentionally scoped so that Steps 1–3 are the primary focus. You are not expected to complete everything during the live session.

A strong result is a clean, well-explained implementation of the core backend flow: domain model → tenant-scoped API → event ingestion → business rule.

Priority	Step	Item	What we are looking for
Must complete	Step 1	Data model design	Clear Tenant → Building → Asset → AssetEvent relationships; reasonable use of TypeScript

Priority	Step	Item	What we are looking for
			types/classes; ability to explain normalization and tenant isolation.
Must complete	Step 2	Simulated tenant context	Tenant context comes from the request in a clear and explicit way: header, query param, or mocked JWT claim.
Must complete	Step 2	GET /buildings	Returns only buildings available to the current tenant.
Must complete	Step 2	POST /assets/:id/events	Ingests a typed and validated asset event; verifies that the asset belongs to the current tenant.
Must complete	Step 3	Business logic rule	Implements or clearly describes the "3 or more error events within 10 minutes → mark asset as critical" rule in a named service-layer function.
Must complete	General	Code structure and explanation	Clear separation between controller/API layer, service/business logic, and storage/model layer; explains trade-offs while coding.
Nice to have	Step 2+	GET /buildings/:id/assets	Returns only assets from a building that belongs to the current tenant.
Nice to have	General	Request/response consistency	Consistent JSON response shape, basic error handling, meaningful status codes.
Nice to have	General	README notes	Short explanation of what was completed, what was skipped, and key design decisions.
Nice to have	General	Database thinking	Brief explanation of how this would map to PostgreSQL/Prisma in production, including tenant-scoped queries and useful indexes.

Priority	Step	Item	What we are looking for
Bonus	Step 4	Real-time layer	Basic WebSocket or SSE implementation, or a clear explanation of how it would work.
Bonus	Step 4	Tenant-safe subscriptions	Clients can only subscribe to buildings that belong to their tenant.
Bonus	Step 5	Minimal React view	Simple functional component that displays assets, current status, and last event timestamp.
Bonus	General	Tests	A few focused tests for the business rule or tenant isolation logic.
Bonus	General	AI usage reflection	Briefly explain how AI tools were used and what was manually verified.

** Categories marked with an asterisk are only scored if attempted.*

Technical Requirements

- TypeScript throughout — no plain JS
- Node.js runtime — NestJS preferred, Express acceptable
- Use in-memory storage by default (plain arrays/maps in the service layer) — no database setup required
- If you want to use PostgreSQL or Prisma, you may — but it is not expected and will not score higher
- No Docker required — local execution is fine
- No authentication implementation required — simulate tenant context explicitly
- AI tools are allowed and encouraged

Deliverables (ZIP Archive)

Your ZIP must contain the following structure:

```
assetpulse/
    ├── src/
    │   └── (your source files)
```

```
├── schema.sql       # only if you used a real DB
├── package.json
├── tsconfig.json
└── README.md        # required
```

README must include:

- How to run the project locally (even if it is just ts-node index.ts)
- What you completed and what you skipped — and why
- At least one design decision you made and your reasoning
- One thing you would do differently with more time

Good luck! Once you're done (or time is up), you can simply leave the call and send your solution to i.pukhov@velvetech.dev